

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 7 PERTEMUAN 2
SEMESTER III



BKPM ACARA 14

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Aplikasi Sistem Informasi Akademik sudah dapat melakukan CRUD data mahasiswa dengan Service Layer dan flash message. Namun aplikasi yang baik bukan hanya yang dapat menyimpan data. Ia juga harus aman dari masukan berbahaya, mampu menangani error tanpa membocorkan informasi teknis kepada pengguna, mencatat error agar programmer dapat menelusurinya, memberi umpan balik yang jelas, dan mencegah form terkirim ulang saat halaman di-refresh.

Masalah yang masih ada sebelum praktikum ini antara lain: error database dapat memunculkan pesan teknis di browser, belum ada catatan (log) ketika error terjadi, rute hapus belum sesuai kebutuhan CRUD lengkap, dan penanganan exception baru berupa pesan umum tanpa pencatatan. Praktikum ini memperbaikinya dengan menerapkan Security, Exception Handling, Logging, Flash Message, dan PRG (Post/Redirect/Get) pada CRUD mahasiswa yang terintegrasi dengan MySQL.

2. Tujuan

1. Menerapkan validasi input dan prepared statement untuk mencegah masukan berbahaya dan SQL Injection.
2. Menerapkan exception handling (try-catch) pada proses CRUD.
3. Membuat logging ke storage/logs/app.log tanpa menampilkan detail error database kepada pengguna.
4. Menerapkan flash message dan pola PRG agar form tidak terkirim ulang saat halaman di-refresh.
5. Melengkapi CRUD mahasiswa yang terintegrasi dengan MySQL dan mengujinya dengan 10 skenario pengujian.

3. Dasar Teori Singkat

Security. Melindungi aplikasi dan database dari masukan atau tindakan yang tidak diinginkan, antara lain melalui validasi input, prepared statement, dan penghilangan informasi teknis dari pesan error.

Prepared Statement. Query ditulis dengan placeholder dan nilai dikirim terpisah, sehingga masukan pengguna selalu diperlakukan sebagai data, bukan perintah SQL, dan membantu mencegah SQL Injection.

Exception Handling. Kode yang berpotensi error dibungkus try-catch. Pengguna cukup menerima pesan umum seperti "Data gagal disimpan", sedangkan detail error tidak ditampilkan.

Logging. Pencatatan kejadian penting atau error ke file (app.log) agar programmer mengetahui kapan error terjadi, error apa, dan di bagian mana. Informasi sensitif seperti password database tidak boleh ditulis ke log.

PRG (Post/Redirect/Get). Setelah form (POST) diproses, pengguna diarahkan (redirect) ke halaman GET. Dengan demikian me-refresh halaman tidak mengirim ulang form.

4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon (Apache + MySQL/MariaDB)
- PHP 8.2
- phpMyAdmin dan file acara_7.sql
- HTML, CSS, Bootstrap
- Git
- Modul Praktikum (BKPM Acara 14)

HASIL PRAKTIK

1. Repository dengan Prepared Statement

Seluruh query pada MahasiswaRepository memakai prepared statement dengan placeholder bernama, termasuk pencarian dengan LIKE dan pengecekan NIM (existsByNim). Pada praktikum ini ditambahkan method all() agar daftar method sesuai dengan langkah 3 modul. Method find(), create(), update(), delete(), dan existsByNim() sudah tersedia dari acara sebelumnya.

```
// Seluruh mahasiswa (BKPM Acara 14, langkah 3: method all()).
public function all(): array
{
    return $this->allWithProdi(null);
}
```

Gambar 2. Method all() pada MahasiswaRepository

Karena emulated prepares dimatikan di Database.php, query dikompilasi lebih dahulu oleh server MySQL. Masukan seperti ' OR '1'='1 hanya diperlakukan sebagai teks biasa.

2. Service: Exception Handling dan Logging

Pada Acara 13, Service hanya menangkap exception di sebagian proses dan tidak mencatatnya. Pada acara ini seluruh proses create, update, dan delete (termasuk validasi yang mengakses database seperti existsByNim) dibungkus try-catch. Exception diteruskan ke handleException() yang mencatat detail ke app.log lalu mengembalikan pesan yang aman untuk pengguna.

```
// ===== CREATE =====
// Seluruh proses dibungkus try-catch (BKPM Acara 14, langkah 5): jika
// database bermasalah, detail error DICATAT ke app.log dan pengguna hanya
// menerima pesan umum "Data gagal disimpan." (tanpa informasi teknis).
public function create(array $input): array
{
    $data = $this->normalize($input);

    try {
        $errors = $this->validate($data);
        if (!empty($errors)) {
            return ['success' => false, 'errors' => $errors];
        }

        $id = $this->repo->create($this->buildEntity($data));
        return ['success' => true, 'errors' => [], 'id' => $id];
    } catch (InvalidArgumentException | PDOException $e) {
        return [
            'success' => false,
            'errors' => $this->handleException($e, 'Gagal menambah data
mahasiswa', 'Data gagal disimpan.'),
        ];
    }
}
```

```

// ===== UPDATE =====
public function update(int $id, array $input): array
{
    $data = $this->normalize($input);

    try {
        if ($id <= 0 || $this->repo->find($id) === null) {
            return [
                'success' => false,
                'not_found' => true,
                'errors' => ['general' => 'Data mahasiswa tidak ditemukan.'],
            ];
        }

        $errors = $this->validate($data, $id);
        if (!empty($errors)) {
            return ['success' => false, 'errors' => $errors];
        }

        $this->repo->update($id, $this->buildEntity($data));
        return ['success' => true, 'errors' => []];
    } catch (InvalidArgumentException | PDOException $e) {
        return [
            'success' => false,
            'errors' => $this->handleException($e, 'Gagal mengubah data
mahasiswa (id=' . $id . ')', 'Data gagal disimpan. '),
        ];
    }
}

// ===== DELETE =====
public function delete(int $id): array
{
    try {
        if ($id <= 0 || $this->repo->find($id) === null) {
            return ['success' => false, 'errors' => ['general' => 'Data mahasiswa
tidak ditemukan. ']];
        }

        $this->repo->delete($id);
        return ['success' => true, 'errors' => []];
    } catch (PDOException $e) {
        return [
            'success' => false,
            'errors' => $this->handleException($e, 'Gagal menghapus data
mahasiswa (id=' . $id . ')', 'Data gagal dihapus. '),
        ];
    }
}

```

Gambar 3. MahasiswaService: create(), update(), dan delete() dengan try-catch

```

// Mencatat exception ke app.log, lalu mengembalikan pesan yang AMAN
// ditampilkan kepada pengguna.
// - InvalidArgumentException : pesan dari Entity (aman, ditulis sendiri)
// - PDOException kode 1062 : duplicate entry -> "NIM sudah terdaftar."
// - PDOException lainnya : pesan umum, detail hanya ada di log
private function handleException(Throwable $e, string $context, string
$userMessage): array
{
    if ($e instanceof InvalidArgumentException) {
        Logger::warning($context, $e);
        return ['general' => $e->getMessage()];
    }
}

```

```

        if ($e instanceof PDOException && ($e->errorInfo[1] ?? null) === 1062) {
            Logger::warning($context . ' (NIM duplikat)', $e);
            return ['nim' => 'NIM sudah terdaftar.'];
        }

        Logger::error($context, $e);
        return ['general' => $userMessage];
    }
}

```

Gambar 4. Method `handleException()`: mencatat error dan menyusun pesan aman

Pembagian pesan: kesalahan validasi ditampilkan sesuai aturannya; kode MySQL 1062 (duplicate entry) diubah menjadi "NIM sudah terdaftar."; seluruh kesalahan database lain hanya menampilkan "Data gagal disimpan." atau "Data gagal dihapus." tanpa detail teknis.

3. Logging ke storage/logs/app.log

Class Logger dibuat pada `app/Core/Logger.php`. Setiap baris log memuat waktu, level (ERROR, WARNING, INFO), konteks kejadian, kelas exception, pesan error, serta nama file dan nomor baris. Tiga keputusan keamanan diterapkan: stack trace tidak dicatat karena dapat memuat argumen fungsi seperti password database pada `new PDO()`, karakter baris baru dibuang agar masukan pengguna tidak dapat menyisipkan baris log palsu, dan log tidak pernah ditampilkan kepada pengguna.

```

final class Logger
{
    public static function path(): string
    {
        return __DIR__ . '/../../storage/logs/app.log';
    }

    public static function error(string $message, ?Throwable $e = null): void
    {
        self::write('ERROR', $message, $e);
    }

    public static function warning(string $message, ?Throwable $e = null): void
    {
        self::write('WARNING', $message, $e);
    }

    public static function info(string $message): void
    {
        self::write('INFO', $message, null);
    }

    private static function write(string $level, string $message, ?Throwable $e): void
    {
        $line = date('Y-m-d H:i:s') . ' - ' . $level . ' - ' . $message;

        if ($e !== null) {
            $line .= ' | ' . get_class($e) . ': ' . $e->getMessage()

```

```

        . ' (' . basename($e->getFile()) . ':' . $e->getLine() . ')';
    }

    $line = str_replace(["\r", "\n"], ' ', $line) . PHP_EOL;

    $dir = dirname(self::path());
    if (!is_dir($dir)) {
        mkdir($dir, 0775, true);
    }

    // Tipe 3 = tambahkan (append) ke file yang ditentukan.
    error_log($line, 3, self::path());
}
}

```

Gambar 5. Class Logger

Contoh format satu baris log (ilustrasi format, bukan hasil pengujian):

```

2026-10-08 10:30:15 - ERROR - Gagal menambah data mahasiswa | PDOException:
SQLSTATE[42S02]: Base table or view not found: 1146 Table 'acara_7.mahasiswa' doesn't
exist (MahasiswaRepository.php:50)

```

Gambar 6. Format baris pada app.log

File app.log dibuat otomatis ketika error pertama terjadi. File ini dimasukkan ke .gitignore (/storage/logs/*.log) karena berisi data runtime, bukan source code.

4. Handler Global, Halaman Error, dan Keamanan Tambahan

Exception yang tidak tertangkap (misalnya MySQL tidak aktif sehingga koneksi gagal sebelum Service dipanggil) ditangani oleh handler global di public/index.php. Handler mencatat error ke app.log lalu menampilkan halaman 500 yang hanya berisi pesan umum. Pengaturan display_errors dimatikan agar PHP tidak menampilkan warning atau error teknis ke browser, dan warning PHP dicatat sebagai WARNING. Header keamanan dasar juga ditambahkan.

```

// ---- Keamanan & penanganan error (BKPM Acara 14) ----
// 1) Error teknis tidak boleh tampil di browser; semuanya dicatat ke app.log.
ini_set('display_errors', '0');
error_reporting(E_ALL);

// 2) Header keamanan dasar.
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: SAMEORIGIN');
header('Referrer-Policy: same-origin');

// 3) Warning/notice PHP dicatat sebagai WARNING.
set_error_handler(function (int $severity, string $message, string $file, int $line):
bool {
    if (!(error_reporting() & $severity)) {
        return false;
    }
    Logger::warning('PHP error: ' . $message . ' (' . basename($file) . ':' . $line .
    ')');
    return true;
});

```

```
// 4) Exception yang lolos dari try-catch (mis. database mati) dicatat, lalu
// pengguna hanya melihat halaman 500 yang aman.
set_exception_handler(function (Throwable $e): void {
    Logger::error('Exception tidak tertangani', $e);
    if (!headers_sent()) {
        http_response_code(500);
    }
    require __DIR__ . '/../app/Views/errors/500.php';
});
```

Gambar 7. Pengaturan keamanan dan handler global pada public/index.php

Praktik keamanan lain yang diterapkan atau dipertahankan pada project:

- Validasi input sisi server pada Service (atribut required di browser mudah dilewati).
- Prepared statement pada seluruh Repository untuk mencegah SQL Injection.
- htmlspecialchars() pada seluruh keluaran teks pengguna untuk mencegah XSS.
- Cookie sesi HttpOnly dan SameSite=Lax, serta session_regenerate_id() saat login untuk mencegah session fixation.
- Aksi yang mengubah data (store, update, delete) hanya tersedia lewat POST dan dilindungi AuthMiddleware.

5. Flash Message dan PRG

Setiap proses POST selalu berakhir dengan redirect, baik saat berhasil maupun saat gagal validasi. Saat berhasil, flash message sukses disimpan di session lalu pengguna diarahkan ke daftar mahasiswa. Saat gagal, input lama, error per kolom, dan flash error disimpan di session lalu pengguna diarahkan kembali ke form. Karena respons terakhir selalu berupa GET, me-refresh halaman tidak mengirim ulang form, dan flash message hanya tampil satu kali.

```
public function store(): void
{
    $result = $this->service->create($_POST);

    if (!$result['success']) {
        $this->backWithErrors($result['errors'], 'mahasiswa/create');
    }

    flash_set('success', 'Data mahasiswa berhasil ditambahkan.');
```

Gambar 8. MahasiswaController::store() menerapkan alur POST, proses, redirect, GET

6. CRUD Lengkap

Halaman CRUD mahasiswa tersedia melalui rute berikut. Rute hapus pada project sebelumnya bernama /mahasiswa/destroy dan diganti menjadi /mahasiswa/delete sesuai modul. Rute hapus hanya tersedia untuk method POST sehingga mengetik URL-nya di browser (GET) menghasilkan 404.

Rute	Method	Operasi	Keterangan
/mahasiswa	GET	Read	Daftar mahasiswa (JOIN prodi) + pencarian nama/NIM
/mahasiswa/create	GET	Create	Form tambah mahasiswa
/mahasiswa/store	POST	Create	Proses tambah lewat Service
/mahasiswa/edit?id=1	GET	Update	Form ubah mahasiswa
/mahasiswa/update	POST	Update	Proses ubah lewat Service
/mahasiswa/delete	POST	Delete	Proses hapus lewat Service (dengan konfirmasi JavaScript)

Tabel 3. Rute CRUD mahasiswa

```
'/mahasiswa/store' => [  
    'controller' => 'MahasiswaController',  
    'action'      => 'store',  
    'middleware' => ['AuthMiddleware'],  
],  
'/mahasiswa/update' => [  
    'controller' => 'MahasiswaController',  
    'action'      => 'update',  
    'middleware' => ['AuthMiddleware'],  
],  
'/mahasiswa/delete' => [  
    'controller' => 'MahasiswaController',  
    'action'      => 'destroy',  
    'middleware' => ['AuthMiddleware'],  
],
```

Gambar 10. Rute POST untuk store, update, dan delete pada routes/web.php

7. Pengujian

Pengujian dilakukan melalui browser pada <http://localhost/si-akademik/public/> menggunakan 10 skenario dari modul. Hasil yang diharapkan:

No	Skenario	Hasil yang diharapkan
1	Tambah mahasiswa dengan data valid	Pesan hijau "Data mahasiswa berhasil ditambahkan."; data muncul di daftar
2	Tambah mahasiswa dengan NIM kosong	Pesan "NIM wajib diisi." dan flash error; data tidak tersimpan

No	Skenario	Hasil yang diharapkan
3	Tambah mahasiswa dengan NIM yang sudah terdaftar	Pesan "NIM sudah terdaftar."; data tidak tersimpan
4	Masukan tidak valid (pengganti email tidak valid): NIM berisi huruf	Pesan "NIM harus berupa angka."
5	Edit data mahasiswa	Pesan hijau "Data mahasiswa berhasil diubah."
6	Hapus data mahasiswa	Konfirmasi muncul, lalu pesan "Data mahasiswa berhasil dihapus."
7	Refresh halaman setelah proses tambah	Tidak ada pengiriman ulang form dan tidak ada data ganda (PRG)
8	Simulasi error database	Pengguna hanya melihat "Data gagal disimpan." (atau halaman 500 aman jika MySQL mati)
9	Periksa app.log	Ada baris ERROR baru berisi waktu, konteks, dan pesan error
10	Periksa pesan yang diterima pengguna	Tidak ada nama tabel, query SQL, atau path file yang tampil di layar

Tabel 4. Skenario pengujian dan hasil yang diharapkan

Simulasi error database (skenario 8) dilakukan dengan mengganti nama tabel mahasiswa sementara di phpMyAdmin (RENAME TABLE mahasiswa TO mahasiswa_tmp) lalu mencoba menambah data, kemudian nama tabel dikembalikan. Variasi lain adalah mematikan MySQL pada XAMPP lalu membuka halaman Data Mahasiswa untuk melihat halaman 500.

The screenshot shows the 'Data Mahasiswa' page of the SI Akademik Polije system. A green success message at the top states 'Data mahasiswa berhasil ditambahkan.' Below this is a table titled 'Daftar Mahasiswa' with 6 data entries. The table columns are NIM, NAMA, PRODI, ANGKATAN, STATUS, and AKSI. The first student listed is Bagas with NIM 2302001, enrolled in Teknik Informatika in 2023, with a status of 'Lulus'. The last student listed is Feri with NIM 2401004, enrolled in Sistem Informasi in 2026, with a status of 'Aktif'. The interface includes a sidebar with navigation links like Dashboard, Data Mahasiswa, Program Studi, Mata Kuliah, and Data Dosen, and a top navigation bar with a login status for 'admin'.

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2302001	Bagas	Teknik Informatika	2023	Lulus	Detail, Edit, Delete
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail, Edit, Delete
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail, Edit, Delete
2401003	Asti	Teknik Informatika	2026	Aktif	Detail, Edit, Delete
2401004	Feri	Sistem Informasi	2026	Aktif	Detail, Edit, Delete

Gambar 11. Tambah data mahasiswa Bernama bagas valid

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Tambah Mahasiswa

Selamat datang, **admin**

NIM wajib diisi.

NIM

NIM wajib diisi.

Nama Lengkap

Farel

Program Studi

Teknik Komputer

Angkatan

2025

Status

Aktif

Gambar 12. NIM kosong

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Tambah Mahasiswa

Selamat datang, **admin**

NIM sudah terdaftar.

NIM

2401001

NIM sudah terdaftar.

Nama Lengkap

Farel

Program Studi

Teknik Komputer

Angkatan

2025

Status

Aktif

Gambar 13. NIM sudah terdaftar

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Tambah Mahasiswa

Selamat datang, admin

NIM harus berupa angka.

NIM

ABC

NIM harus berupa angka.

Nama Lengkap

Farel

Program Studi

Teknik Komputer

Angkatan

2025

Status

Aktif

Gambar 14. Masukan tidak valid

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Data Mahasiswa

Selamat datang, admin

Daftar mahasiswa yang terdaftar

Daftar Mahasiswa

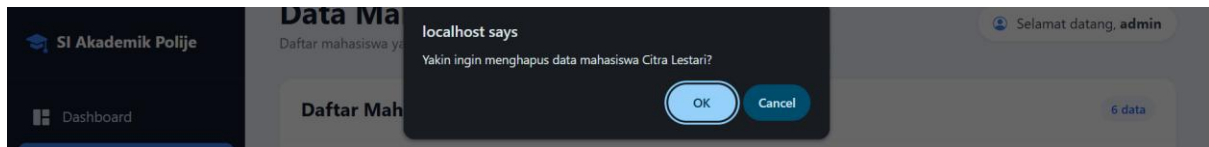
6 data

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2302001	Bagas Pradana	Teknik Informatika	2023	Lulus	Detail Edit Hapus
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail Edit Hapus
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail Edit Hapus
2401003	Asti	Teknik Informatika	2026	Aktif	Detail Edit Hapus
2401004	Feri	Sistem Informasi	2026	Aktif	Detail Edit Hapus
2402001	Citra Lestari	Sistem Informasi	2026	Aktif	Detail Edit Hapus

Gambar 15. Edit data mahasiswa Bernama Bagas menjadi Bagas Pradana



SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Data Mahasiswa

Daftar mahasiswa yang terdaftar

Selamat datang, admin

✓ Data mahasiswa berhasil dihapus.

Daftar Mahasiswa

5 data

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2302001	Bagas Pradana	Teknik Informatika	2023	Lulus	Detail
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail
2401003	Asti	Teknik Informatika	2026	Aktif	Detail
2401004	Feri	Sistem Informasi	2026	Aktif	Detail

Gambar 16. Hapus data mahasiswa

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Data Mahasiswa

Daftar mahasiswa yang terdaftar

Selamat datang, admin

✓ Data mahasiswa berhasil ditambahkan.

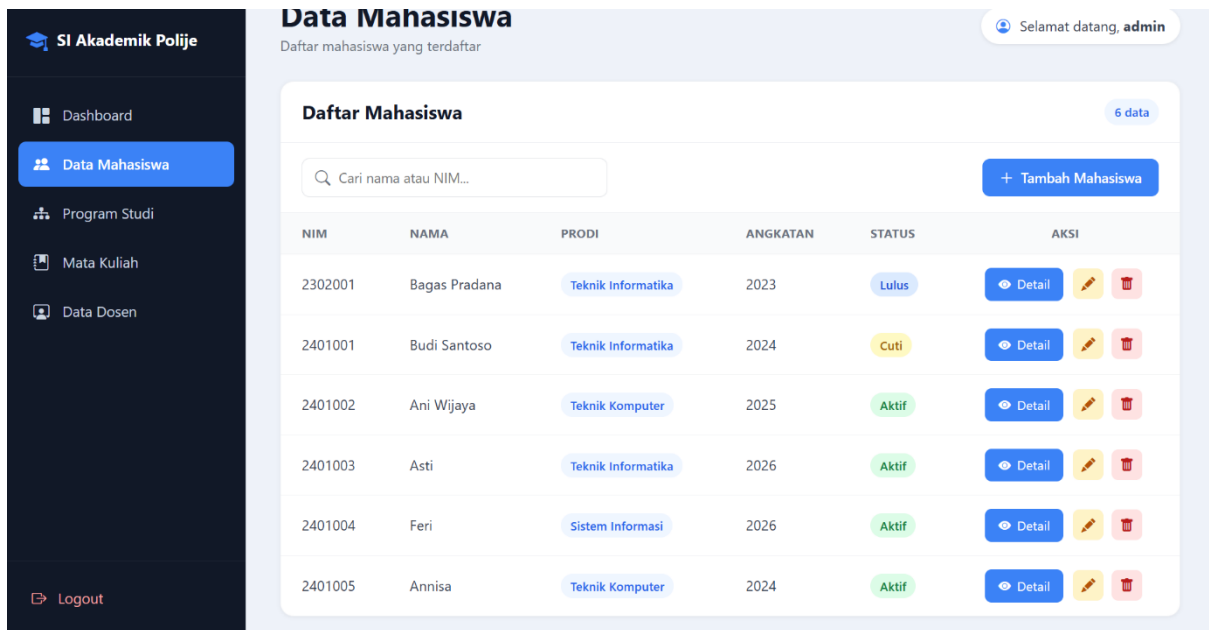
Daftar Mahasiswa

6 data

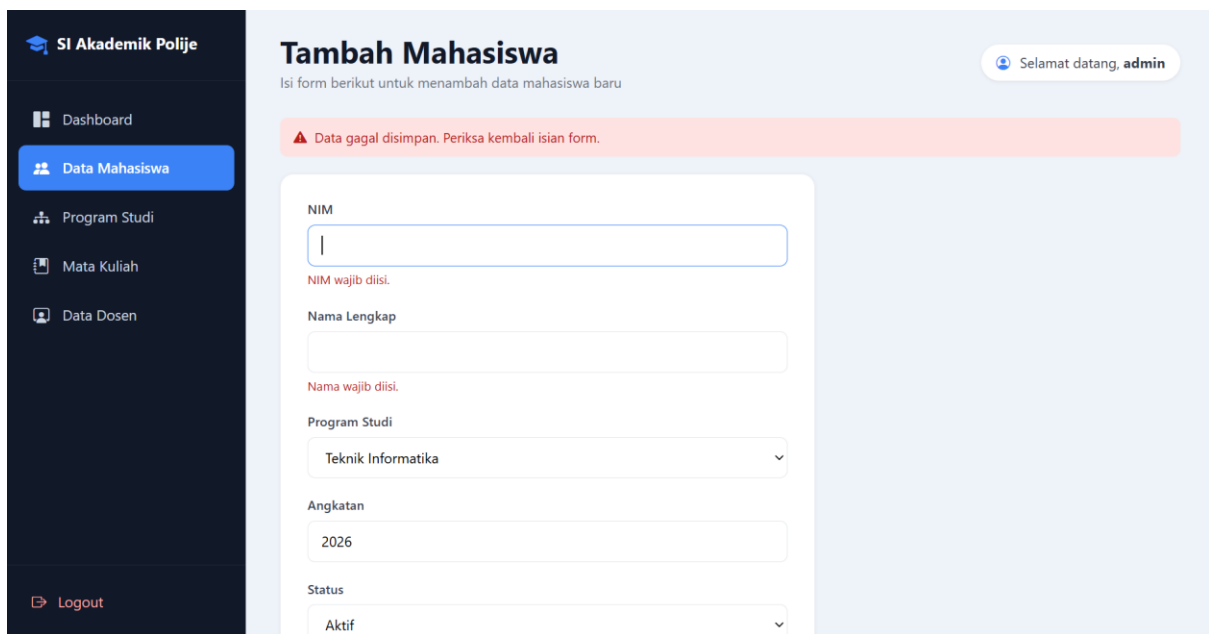
Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2302001	Bagas Pradana	Teknik Informatika	2023	Lulus	Detail
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail
2401003	Asti	Teknik Informatika	2026	Aktif	Detail
2401004	Feri	Sistem Informasi	2026	Aktif	Detail



Gambar 17. Refresh setelah proses tambah (PRG)



Gambar 18. Pesan aman saat error database



Gambar 19. Isi app.log setelah simulasi error

8. Ringkasan Output Praktikum

Output yang diminta	Hasil
Aplikasi CRUD Mahasiswa	Create, Read, Update, Delete mahasiswa (Bagian 8)
Database MySQL	Database acara_7 dengan koneksi PDO (Bagian 1 dan 2)
Repository dengan Prepared Statement	MahasiswaRepository (Bagian 3)
Service dengan validasi dan Business Logic	MahasiswaService (Bagian 4)
Exception Handling	try-catch pada Service dan handler global (Bagian 4 dan 6)
File app.log	storage/logs/app.log dari Logger (Bagian 5)
Flash Message	Pesan sukses dan error (Bagian 7)
Implementasi PRG	Redirect setelah setiap POST (Bagian 7)
Repository GitHub berisi source code	Repository si-akademik (Bagian 10)

Tabel 5. Ringkasan output praktikum

9. Analisis dan Pembahasan

Pada Acara 13, exception hanya ditangkap di Service untuk sebagian proses, dan kesalahan database tidak meninggalkan jejak. Akibatnya programmer tidak dapat mengetahui penyebab "Data gagal disimpan". Dengan Logger, setiap kegagalan memiliki catatan waktu, konteks, dan pesan error, sementara pengguna tetap hanya melihat pesan umum. Pemisahan ini penting karena pesan database dapat membocorkan nama tabel, struktur query, atau lokasi file yang bisa dimanfaatkan penyerang.

Pencatatan stack trace sengaja dihindari. Pada konfigurasi PHP tertentu, trace memuat argumen fungsi, sehingga kegagalan new PDO(...) dapat ikut menulis username dan password database ke log. Dengan hanya mencatat kelas, pesan, nama file, dan nomor baris, log tetap berguna tanpa memuat informasi sensitif. Pembuangan karakter baris baru melindungi log dari log injection ketika pesan error memuat masukan pengguna, misalnya nilai pada pesan Duplicate entry.

Ada dua jalur penanganan error. Kesalahan yang terjadi pada proses tulis ditangkap di Service sehingga pengguna kembali ke form dengan flash message. Kesalahan yang terjadi sebelum Service sempat bekerja, misalnya MySQL mati, ditangkap handler global dan menampilkan halaman 500. Dua lapisan ini memastikan tidak ada jalur yang menampilkan error teknis ke browser.

PRG diterapkan pada semua hasil POST, termasuk saat validasi gagal. Tanpa PRG, me-refresh halaman setelah menambah data akan memunculkan dialog kirim ulang form dan dapat membuat data ganda. Keamanan aplikasi juga masih dapat dikembangkan lebih lanjut, misalnya dengan token CSRF pada setiap form, pembatasan percobaan login, dan penyimpanan password ter-hash pada tabel pengguna menggantikan akun hardcode.

KESIMPULAN

1. Validasi di Service dan prepared statement di Repository melindungi CRUD mahasiswa dari masukan tidak valid dan SQL Injection.
2. Exception handling dengan try-catch dan handler global memastikan pengguna hanya menerima pesan error yang aman, tanpa informasi teknis database.
3. Logger mencatat detail error ke storage/logs/app.log tanpa stack trace dan tanpa informasi sensitif, sehingga programmer dapat menelusuri penyebab error.
4. Flash message dan pola PRG memberikan umpan balik yang jelas serta mencegah form terkirim ulang saat halaman di-refresh.
5. CRUD mahasiswa lengkap dan terintegrasi dengan MySQL (database acara_7) berhasil dibuat, dan pekerjaan dicatat pada branch fitur-keamanan-logging yang di-merge ke main.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.